

Entregable 1 - Capacitación Reactivo

La resolución de este entregable es obligatorio dentro de la capacitación de programación reactiva. La fecha de inicio del entregable es el 25/03/2025 y su fecha límite es el 03/04/2025.

El entregable consiste de dos partes, una teórica y otra práctica. Cada alumno de la capacitación deberá entregar un archivo con las respuestas del teórico y un link al repositorio donde resolvió la parte práctica. Este archivo debe subirse antes de la fecha límite en el directorio de drive asignado a cada uno.

Para todos los ejercicios prácticos, se espera que se incluyan logs a través de logback y las herramientas que vimos en clase (**los logs deben estar dentro de callbacks y se espera que dichos logs ocurran en casos exitosos, error, vacíos, completitud, etc.**). Para verificar que esté andando correctamente la solución, se pueden ejecutar los tests del repositorio.

El código base para la resolución del entregable es el siguiente:

<https://gitlab.com/tenpo/development/capacitacion-reactivo>

Teórico

1. Dados los siguientes callbacks: `doOnNext`, `doOnError`, `doOnComplete`, `doFirst`, `doFinally`, `doOnSubscribe`, `doOnTerminate`. Ordenar en orden cronológico por cómo se verían los logs en pantalla en base a cómo ejecutaría un código con ellos. Explicar por qué se da ese orden.
2. Basado en el ejercicio 2c del práctico: ¿qué ocurriría si el `findAllCountries` arrojara un error antes de emitir el valor "Francia"? ¿Cómo se podría hacer para garantizar que se emita hasta llegar a "Francia" sin usar un `onErrorContinue`?
3. ¿Qué estrategias de backpressure puede utilizarse con Java Reactor? ¿Cuál es la estrategia predeterminada que utiliza Reactor?

Práctico

1. Resolver este ejercicio en la clase **tasks.task1.T1Question1**.
 - a. Utilizando el método `findFirstName` del `UserService`, realizar una lógica que procese un único nombre para mapearlo a su longitud de caracteres en caso de que el nombre comience con la letra A. En su defecto, el suscriptor debe recibir el valor -1.
 - b. Utilizando los métodos `findFirstName`, `existsByName`, `update` e `insert` del `UserService`, realizar una lógica que actualice el registro en la BD si ya existe. Si no existe, debe insertarlo.
 - c. Utilizando el método `findFirstByName` del `UserService`, devolver el valor retornado por el servicio. Si el servicio no devuelve nada, retornar un

ResourceNotFoundException. En caso de que el servicio arroje un error, arrojar una UserServiceException.

2. Resolver este ejercicio en la clase **tasks.task1.T1Question2**
 - a. Utilizando el método findAllCountries del CountryService, realizar una lógica que devuelva los primeros 5 países distintos.
 - b. Utilizando el método findAllCountries del CountryService, realizar una lógica que emita países hasta que encuentre a "Argentina".
 - c. Utilizando el método findAllCountries del CountryService, realizar una lógica que emita países siempre y cuando no encuentre a "Francia".
3. Resolver este ejercicio en la clase **tasks.task1.T1Question3**
 - a. Considerando que se recibe un objeto de tipo Page<String>, realizar una lógica para emitir todos los elementos internos de la página. Tener en cuenta que el objeto page puede ser nulo.
 - b. Utilizando el método findCurrenciesByCountry del CountryService, realizar una lógica que emita todas las distintas monedas asociadas al país que llega por parámetro.
 - c. Utilizando el método findAllCountries del CountryService y el translate del TranslatorService, traducir los nombres de los 3 primeros países. Tener en cuenta que el TranslatorService puede retornar null si no sabe traducir el país que recibe por parámetro.
4. Resolver este ejercicio en la clase **tasks.task1.T1Question4**
 - a. Asumiendo que el método findAllCountries es costoso de ejecutar, consumir dicho método del CountryService con el fin de calcular la cantidad de repeticiones por país en un lote de 200 países emitidos, de manera **asíncronica** (el resultado debe verse por consola en forma de mapa). Para el mismo lote de 200 países, se debe retornar el listado de países en orden alfabético sin repetidos.
5. Resolver este ejercicio en la clase **tasks.task1.T1Question5**
 - a. Utilizando el método calculate del CalculatorService, aplicarlo a un rango de números a partir del 100 hasta el 1000. El cálculo puede devolver un resultado, un vacío o un error. Si y solo si no hay errores, proceder a retornar un nombre al azar a través del UserService (findFirstName), de lo contrario debe retornar "Chuck Norris". **El cálculo debe aplicarse a los números en orden ascendiente.**
 - b. Utilizando el método calculate del CalculatorService, aplicarlo a un rango de números a partir del 100 hasta el 1000. El cálculo puede devolver un resultado, un vacío o un error. Si y solo si no hay errores, proceder a retornar los primeros 3 nombres sin importar repetidos a través del UserService (findAllNames), de lo contrario debe completarse el publisher. **El cálculo debe aplicarse a los números sin orden alguno.**
6. Resolver este ejercicio en la clase **tasks.task1.T1Question6**
 - a. Desarrollar un publisher que emita el precio de una acción en vivo cada 500ms. *Consideraciones:* El precio solo puede moverse entre 1 y 500. Configurar el publisher para que los suscriptores cuando se conecten vean la data en tiempo real, perdiendo todos los precios ya emitidos por el publisher antes que estos se conecten.
7. Resolver este ejercicio en la clase **tasks.task1.T1Question7**

- a. Desarrollar una lógica para generar un `UserAccount` obteniendo la información de los siguientes servicios `UserService.getUserById()` y `AccountService.getAccountByUserId()`.
8. Resolver este ejercicio en la clase **tasks.task1.T1Question8**
- a. Suscribirse al servicio `TransactionService.authorizeTransaction()`, teniendo en cuenta las siguientes condiciones. Consideraciones:
 - i. Si el servicio tarda más de 3 segundos en enviarnos la respuesta, capturar el timeout y convertirlo en `AuthorizationTimeoutException`.
 - ii. Establecer una estrategia de 3 reintentos cada 500ms para cualquier error del servicio excepto los Timeouts.
 - iii. Capturar todas las excepciones que devuelva el servicio y transformarlas en una `PaymentProcessingException` (excluyendo los timeouts, que ya los mapeamos en `AuthorizationTimeoutException`).